

# Streaming API Responses

A Complete Guide for AI Engineers — Basic to Advanced

Learn how to stream real-time token-by-token responses from OpenAI's API using Python — from your first chunk to production-ready async endpoints.

|                                                     |                                                   |                                                          |
|-----------------------------------------------------|---------------------------------------------------|----------------------------------------------------------|
| <b>Server-Sent Events SSE</b><br>protocol deep dive | <b>8 Code Examples</b> Basic →<br>async → FastAPI | <b>Tool Call Streaming</b> Function<br>args in real time |
|-----------------------------------------------------|---------------------------------------------------|----------------------------------------------------------|

# Table of Contents

---

|    |                                                 |    |
|----|-------------------------------------------------|----|
| 1. | What Is Streaming and Why Use It?               | 3  |
| 2. | How Streaming Works — The SSE Protocol          | 3  |
| 3. | Your First Streaming Call                       | 4  |
| 4. | Anatomy of a Chunk                              | 4  |
| 5. | Accumulating Content and Checking Finish Reason | 5  |
| 6. | The Stream Context Manager (.stream())          | 5  |
| 7. | Streaming Tool / Function Calls                 | 6  |
| 8. | Async Streaming                                 | 7  |
| 9. | Building a FastAPI Streaming Endpoint           | 8  |
| 10 | Error Handling and Retries                      | 9  |
| .  |                                                 |    |
| 11 | Tracking Token Usage During Streaming           | 9  |
| .  |                                                 |    |
| 12 | The Responses API (2025+)                       | 10 |
| .  |                                                 |    |
| 13 | Best Practices Cheat Sheet                      | 10 |
| .  |                                                 |    |

## 1. What Is Streaming and Why Use It?

By default, the OpenAI API waits until the model finishes generating every token, then sends the complete response in one HTTP response. For short answers this is fine, but for long outputs — code generation, essays, reports — the user stares at a blank screen for several seconds before anything appears.

Streaming flips this model: tokens are sent to your application the moment the model produces them, just like a human typing in real time. Your app can render each word as it arrives, making the interaction feel **instant**.

### Non-streaming vs. Streaming

| Non-Streaming            | Streaming                  |
|--------------------------|----------------------------|
| Waits for full response  | Tokens arrive in real time |
| Single HTTP response     | Chunked transfer / SSE     |
| Higher perceived latency | Lower perceived latency    |
| Simple to parse          | Requires chunk iteration   |
| Good for short outputs   | Essential for long outputs |

## 2. How Streaming Works — The SSE Protocol

OpenAI uses **Server-Sent Events (SSE)**, a standard web protocol where the server keeps an HTTP connection open and pushes data chunks to the client. Each chunk is a plain-text line in this format:

#### Raw SSE wire format

```
data: {"id":"chatcmpl-abc","choices":[{"delta":{"content":"Hello"},...}],...}

data: {"id":"chatcmpl-abc","choices":[{"delta":{"content":" world"},...}],...}

data: [DONE]
```

Each **data:** line contains a JSON object. The very last event is the sentinel data: `[DONE]`, which signals the stream is finished. The OpenAI Python SDK handles all of this parsing for you — you just iterate chunks.

■ **Note:** You never need to parse SSE manually when using the official openai Python library. The SDK handles chunked decoding, JSON parsing, and the [DONE] sentinel automatically.

### 3. Setup and Your First Streaming Call

---

Install the library and initialise the client:

```
setup.py

# Install the OpenAI Python library
# pip install openai

import openai

client = openai.OpenAI(api_key="YOUR_OPENAI_API_KEY")
# Or set the OPENAI_API_KEY environment variable and call openai.OpenAI()
```

Now compare a non-streaming call with a streaming one:

**no\_streaming.py — full response arrives at once**

```
# Non-streaming: waits for the FULL response before returning
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Write a short poem about Python."}]
)

# The entire text arrives at once
print(response.choices[0].message.content)
```

**basic\_streaming.py — tokens arrive one by one**

```

# Streaming: tokens arrive one chunk at a time
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Write a short poem about Python."}],
    stream=True, # <-- this enables streaming
)

# Iterate over each chunk as it arrives
for chunk in stream:
    # Each chunk has a list of 'choices'
    delta = chunk.choices[0].delta

    # delta.content holds the new token(s); None when there's nothing yet
    if delta.content is not None:
        print(delta.content, end="", flush=True)

print() # newline after streaming finishes

```

The only change is adding `stream=True`. The return value changes from a `ChatCompletion` object to an iterator of `ChatCompletionChunk` objects.

## 4. Anatomy of a Chunk

Every chunk is a `ChatCompletionChunk` object. Understanding its structure is key to working with streaming correctly.

chunk\_anatomy.py — inspect the raw chunk

```
# Inspect the raw chunk object to understand its structure
for chunk in stream:
    print(chunk)
    # Typical output:
    # ChatCompletionChunk(
    #   id='chatcmpl-abc123',
    #   choices=[
    #     Choice(
    #       delta=ChoiceDelta(content='Hello', role=None, tool_calls=None),
    #       finish_reason=None,
    #       index=0,
    #       logprobs=None
    #     )
    #   ],
    #   created=1715000000,
    #   model='gpt-4o-2024-05-13',
    #   object='chat.completion.chunk',
    #   usage=None      # <-- usage is None mid-stream
    # )
    #
    # The LAST chunk has finish_reason set and content='' or None:
    # Choice(delta=ChoiceDelta(content=None), finish_reason='stop', ...)
```

| Field                                 | Type                    | Description                                                                             |
|---------------------------------------|-------------------------|-----------------------------------------------------------------------------------------|
| <code>id</code>                       | <code>str</code>        | Same across all chunks of one completion                                                |
| <code>choices[0].delta.content</code> | <code>str   None</code> | The new token(s). None on first and last chunk.                                         |
| <code>choices[0].delta.role</code>    | <code>str   None</code> | 'assistant' on first chunk only, None after.                                            |
| <code>choices[0].finish_reason</code> | <code>str   None</code> | None mid-stream; 'stop', 'length', or 'tool_calls' on last chunk.                       |
| <code>usage</code>                    | <code>None</code>       | None mid-stream. Populated on last chunk only if <code>stream_options</code> requested. |

## 5. Accumulating Content and Checking Finish Reason

In many applications you need both: print tokens as they arrive AND have the complete text when the stream ends. The pattern below collects chunks into a string.

accumulate.py

```
# Pattern: accumulate all chunks into a single string
collected_chunks = []
full_text = ""

stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "List 3 benefits of streaming APIs."}],
    stream=True,
)

for chunk in stream:
    delta_content = chunk.choices[0].delta.content
    if delta_content is not None:
        full_text += delta_content
        collected_chunks.append(delta_content)
        print(delta_content, end="", flush=True)

print()
print(f"\nTotal characters received: {len(full_text)}")
print(f"Total chunks received: {len(collected_chunks)}")
```

finish\_reason.py — always handle the final chunk

```
# Always check finish_reason to know WHY the stream stopped
for chunk in stream:
    delta = chunk.choices[0].delta
    reason = chunk.choices[0].finish_reason

    if delta.content:
        print(delta.content, end="", flush=True)

    if reason is not None:
        print(f"\n\n[Stream ended – finish_reason: {reason}]")
        # Possible values:
        # 'stop' → model finished naturally
        # 'length' → hit max_tokens limit
        # 'tool_calls' → model called a tool/function
        # 'content_filter' → content was filtered
        # 'function_call' → deprecated, use tool_calls
```

## 6. The Stream Context Manager (.stream())

The SDK exposes a higher-level `.stream()` helper that wraps the low-level iterator inside a context manager. It provides:

- **text\_stream** — yields plain string content, skipping empty deltas automatically.
- **get\_final\_completion()** — returns the fully-typed, completed `ChatCompletion` after the loop.
- **Automatic cleanup** — the connection is properly closed even if you break out of the loop early.

stream\_manager.py — preferred pattern for most use cases

```
# Use the stream as a context manager for automatic cleanup
with client.chat.completions.stream(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Explain async I/O in Python."}],
) as stream:
    for text in stream.text_stream:
        # text_stream yields only the string content — no need to check .delta
        print(text, end="", flush=True)

# After the 'with' block, get the fully-typed final completion:
final = stream.get_final_completion()
print(f"\n\nFinish reason : {final.choices[0].finish_reason}")
print(f"Prompt tokens : {final.usage.prompt_tokens}")
print(f"Output tokens : {final.usage.completion_tokens}")
```

■ **Note:** Use `.stream()` (the context manager) for new code. Reserve `stream=True` (the raw iterator) only when you need low-level access to every chunk field.

## 7. Streaming Tool / Function Calls

When the model decides to call a tool, the function name and JSON arguments arrive in streaming chunks too — they don't wait for the full JSON to be ready. You must buffer the argument fragments and parse them once `finish_reason == 'tool_calls'`.

tool\_streaming.py — assemble function arguments from delta chunks



```

import json

tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"},
                "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["city"]
        }
    }
}]

tool_call_id = None
function_name = None
args_buffer = ""

with client.chat.completions.stream(
    model="gpt-4o",
    messages=[{"role": "user", "content": "What is the weather in Tokyo?"}],
    tools=tools,
) as stream:
    for chunk in stream:
        delta = chunk.choices[0].delta

        # Tool call chunks arrive through delta.tool_calls
        if delta.tool_calls:
            tc = delta.tool_calls[0]

            # First chunk sets the id and function name
            if tc.id:
                tool_call_id = tc.id
            if tc.function.name:
                function_name = tc.function.name

            # Subsequent chunks stream the JSON arguments
            if tc.function.arguments:
                args_buffer += tc.function.arguments
                print(tc.function.arguments, end="", flush=True)

        reason = chunk.choices[0].finish_reason
        if reason == "tool_calls":
            args = json.loads(args_buffer)
            print(f"\n\nTool called : {function_name}")
            print(f"Arguments : {json.dumps(args, indent=2)}")

```

■■ **Warning:** Always JSON-parse `args_buffer` only after the stream ends (`finish_reason='tool_calls'`). Intermediate chunks contain partial JSON that is not yet valid.

## 8. Async Streaming

---

For web servers, chatbots, and any I/O-bound application you should use AsyncOpenAI combined with `async for`. This keeps your event loop unblocked while waiting for tokens.

async\_streaming.py — non-blocking streaming with asyncio

```
import asyncio
import openai

async_client = openai.AsyncOpenAI(api_key="YOUR_API_KEY")

async def stream_response(prompt: str) -> str:
    """Stream a response and return the full text."""
    full_text = ""

    async with async_client.chat.completions.stream(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        async for text in stream.text_stream:
            print(text, end="", flush=True)
            full_text += text

    print() # newline
    return full_text

async def stream_multiple(prompts: list[str]):
    """Stream multiple completions CONCURRENTLY."""
    tasks = [stream_response(p) for p in prompts]
    results = await asyncio.gather(*tasks)
    return results

# Run it
asyncio.run(stream_response("What is the event loop in Python?"))
```

Using `asyncio.gather()` lets you stream from multiple prompts truly concurrently — all requests run in parallel on the same event loop.

## 9. Building a FastAPI Streaming Endpoint

---

The most common production pattern is a backend API that streams OpenAI tokens directly to a browser or mobile client via SSE. FastAPI's `StreamingResponse` makes this straightforward.

main.py — FastAPI SSE streaming endpoint

```

# pip install fastapi uvicorn openai
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import openai
import asyncio

app = FastAPI()
client = openai.AsyncOpenAI(api_key="YOUR_API_KEY")

async def token_generator(prompt: str):
    """Yield SSE-formatted chunks for the browser."""
    async with client.chat.completions.stream(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        async for text in stream.text_stream:
            # Server-Sent Event format: 'data: <payload>\n\n'
            yield f"data: {text}\n\n"

        yield "data: [DONE]\n\n"

@app.get("/stream")
async def stream_endpoint(prompt: str):
    return StreamingResponse(
        token_generator(prompt),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no", # Disable nginx buffering
        },
    )

# Run: uvicorn main:app --reload
# Test: curl "http://localhost:8000/stream?prompt=Hello+world"

```

The browser consumes this with the native EventSource API or the `@microsoft/fetch-event-source` library for POST support. The header `X-Accel-Buffering: no` is critical when running behind nginx — without it, nginx buffers the entire response before forwarding.

## 10. Error Handling and Retries

---

Streaming connections can fail mid-way through a response. Always wrap streaming calls in error handling, especially for rate limits (429) and transient network errors.

error\_handling.py — retry with exponential backoff

```
import openai
import time

def stream_with_retry(prompt: str, max_retries: int = 3) -> str:
    """Stream with exponential backoff on transient errors."""
    for attempt in range(max_retries):
        try:
            full_text = ""
            with client.chat.completions.stream(
                model="gpt-4o",
                messages=[{"role": "user", "content": prompt}],
                timeout=30.0, # seconds – applies to the whole stream
            ) as stream:
                for text in stream.text_stream:
                    print(text, end="", flush=True)
                    full_text += text
            print()
            return full_text

        except openai.APIConnectionError as e:
            print(f"\nConnection error (attempt {attempt+1}): {e}")
        except openai.RateLimitError as e:
            wait = 2 ** attempt
            print(f"\nRate limited – retrying in {wait}s...")
            time.sleep(wait)
        except openai.APIStatusError as e:
            print(f"\nAPI error {e.status_code}: {e.message}")
            break # Don't retry on 4xx
        except Exception as e:
            print(f"\nUnexpected error: {e}")
            break

    return ""

result = stream_with_retry("Explain Python generators in two sentences.")
```

## 11. Tracking Token Usage During Streaming

---

Token usage is not included in streaming chunks by default. To receive it, pass `stream_options={'include_usage': True}`. Usage appears only on the final chunk where

`chunk.usage` is not `None`.

token\_usage.py — get usage stats at end of stream

```
# Track token usage — only available in the FINAL chunk
# when stream_options={'include_usage': True} is set
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "What is machine learning?"}],
    stream=True,
    stream_options={"include_usage": True}, # <-- request usage stats
)

usage = None
for chunk in stream:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end="", flush=True)

    # Usage only appears on the very last chunk
    if chunk.usage is not None:
        usage = chunk.usage

print()
if usage:
    print(f"\nPrompt tokens    : {usage.prompt_tokens}")
    print(f"Completion tokens: {usage.completion_tokens}")
    print(f"Total tokens      : {usage.total_tokens}")
```

## 12. The Responses API (2025+)

---

OpenAI introduced the **Responses API** as a successor to Chat Completions. It uses a typed event model instead of raw delta chunks, making it easier to handle different event types (text, tool calls, errors) in a structured way.

responses\_api.py — newer streaming API with typed events

```

# The newer Responses API (2025+) – supports streaming natively
# Uses client.responses.create() instead of client.chat.completions.create()

with client.responses.stream(
    model="gpt-4o",
    input="Explain streaming in one paragraph.",
) as stream:
    for event in stream:
        # The Responses API emits typed events
        if event.type == "response.output_text.delta":
            print(event.delta, end="", flush=True)
        elif event.type == "response.completed":
            print(f"\n\nTotal tokens: {event.response.usage.total_tokens}")

# Shorthand: iterate text only
with client.responses.stream(
    model="gpt-4o",
    input="Name 5 Python libraries for AI.",
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

```

■ **Note:** As of 2025, the Responses API is available on gpt-4o and gpt-4o-mini. The Chat Completions API continues to be fully supported alongside it.

## 13. Best Practices Cheat Sheet

---

| Situation                      | Recommendation                                                                                                       |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Simple scripts / notebooks     | Use <code>stream=True</code> with a simple for loop.                                                                 |
| Production apps / APIs         | Use <code>.stream()</code> context manager with <code>AsyncOpenAI</code> .                                           |
| Web endpoints (REST/SSE)       | Use <code>FastAPI StreamingResponse</code> + <code>text/event-stream</code> content type.                            |
| Need full text after streaming | Call <code>stream.get_final_completion()</code> after the with block.                                                |
| Streaming tool calls           | Buffer <code>delta.tool_calls[0].function.arguments</code> ; parse JSON on <code>finish_reason='tool_calls'</code> . |
| Token counting                 | Set <code>stream_options={'include_usage': True}</code> ; read <code>chunk.usage</code> on the last chunk.           |
| Error resilience               | Catch <code>APIConnectionError</code> + <code>RateLimitError</code> ; retry with exponential backoff.                |
| Nginx / reverse proxy          | Set <code>X-Accel-Buffering: no</code> header to prevent buffering.                                                  |
| Breaking early from stream     | Use <code>.stream()</code> context manager — it closes the connection safely on break.                               |
| Concurrent streams             | Use <code>asyncio.gather()</code> with <code>AsyncOpenAI</code> for parallel requests.                               |

---

Reference: OpenAI Platform Docs — [platform.openai.com/docs/guides/streaming-responses](https://platform.openai.com/docs/guides/streaming-responses) | openai Python SDK — [github.com/openai/openai-python](https://github.com/openai/openai-python)